EXP-8

Student Name: Srijan Tripathi **UID:** 24BAI70081

Branch: AIT-CSE (AIML) **Section/Group:** 24AIT_KRG-1A

Semester: 5 **Subject Code:** 24CSP-310

Subject Name : Advanced Database Management System

A)

A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

The company wants to provide an order report to an external client. The report must contain the **Order ID, Order Date, Customer Name, Product Name, and Final Amount**.
For every order, first calculate the **Gross Amount** using:
Gross Amount = Unit Price × Quantity
Then calculate the **Discount Amount** using:
Discount Amount = Gross Amount × Discount Percentage / 100

Finally, calculate the **Final Amount** using:

Final Amount = Gross Amount – Discount Amount

Create a **complex view** using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.
The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user **only SELECT permission on the view**. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

CODE:






```sql
CREATE TABLE customer_master
(
    customer_id VARCHAR(5) PRIMARY KEY,
    full_name VARCHAR(50) NOT NULL,
    city VARCHAR(30)
);

CREATE TABLE product_catalog
(
    product_id VARCHAR(5) PRIMARY KEY,
    product_name VARCHAR(50) NOT NULL,
    unit_price NUMERIC(10,2) NOT NULL
);

CREATE TABLE sales_orders
(
    order_id SERIAL PRIMARY KEY,
    product_id VARCHAR(5)
        REFERENCES product_catalog(product_id),

    quantity INT NOT NULL,

    customer_id VARCHAR(5)
        REFERENCES customer_master(customer_id),

    discount_percent NUMERIC(5,2),

    order_date DATE NOT NULL
);
```

```sql
INSERT INTO customer_master
(customer_id, full_name, city)
VALUES
('C1', 'Amit Sharma', 'Delhi'),
('C2', 'Priya Verma', 'Mumbai'),
('C3', 'Ravi Kumar', 'Bangalore'),
('C4', 'Neha Singh', 'Kolkata'),
('C5', 'Arjun Mehta', 'Hyderabad');

INSERT INTO product_catalog
(product_id, product_name, unit_price)
VALUES
('P1', 'Smartphone X100', 25000),
('P2', 'Laptop Pro 15', 65000),
('P3', 'Wireless Earbuds', 5000),
('P4', 'Smartwatch Fit', 30000),
('P5', 'Gaming Console', 45000);
```






```sql
INSERT INTO sales_orders
(product_id, quantity, customer_id, discount_percent, order_date)
VALUES
('P1', 2, 'C1', 5, '2025-09-01'),
('P2', 1, 'C2', 10, '2025-09-02'),
('P3', 3, 'C3', 0, '2025-09-03'),
('P4', 1, 'C4', 8, '2025-09-04'),
('P5', 1, 'C5', 12, '2025-09-05'),
('P1', 1, 'C2', 5, '2025-09-06'),
('P3', 2, 'C1', 0, '2025-09-07');
```

OUTPUT SCREENSHOTS:

	order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	final_amount numeric
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.0000000000000000
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.0000000000000000
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.0000000000000000000
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.0000000000000000
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.0000000000000000
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.0000000000000000
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.0000000000000000000

B)

An e-commerce company wants to regularly analyze the performance of its **product categories**.
The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple **JOINs, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements**.
Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.
Create a PostgreSQL **Materialized View** named: mv_category_sales

The report must contain:

- Category ID
- Category Name
- Total Products Sold
- Total Orders
- Unique Customers
- Total Revenue
- Average Revenue per Order
- Category Performance

Also demonstrate that when new order data is inserted, the Materialized View does **not automatically reflect the change** and must be refreshed.

Finally, use EXPLAIN ANALYZE to compare the performance of the original analytical query with the Materialized View query.

Tasks:

Task	Description
1	Create the required source tables.
2	Insert the given category, product, customer, and order data.
3	Write a query to generate the category-wise sales report.
4	Execute the query using `EXPLAIN ANALYZE`.
5	Create the Materialized View `mv_category_sales`.
6	Display the data from the Materialized View.
7	Execute `EXPLAIN ANALYZE` on the Materialized View query.
8	Compare the execution time of both queries.
9	Insert a new order and order detail.
10	Query the Materialized View before refreshing it and observe the stale data.
11	Refresh the Materialized View.
12	Query the Materialized View again and verify the updated result.

CODE:

Task 1:

```sql
CREATE TABLE category_master (
    category_id VARCHAR(10) PRIMARY KEY,
    category_name VARCHAR(50)
);

CREATE TABLE product_master (
    product_id VARCHAR(10) PRIMARY KEY,
    product_name VARCHAR(50),
    category_id VARCHAR(10) REFERENCES category_master(category_id)
);

CREATE TABLE customer_master (
    customer_id VARCHAR(10) PRIMARY KEY,
    customer_name VARCHAR(50)
);

CREATE TABLE order_details (
    order_id VARCHAR(10),
    customer_id VARCHAR(10) REFERENCES customer_master(customer_id),
    product_id VARCHAR(10) REFERENCES product_master(product_id),
    quantity INT,
    unit_price NUMERIC
);
```

Task 2:

```sql
INSERT INTO category_master (category_id, category_name) VALUES
('C1', 'Electronics'),
('C2', 'Clothing'),
('C3', 'Books');

INSERT INTO product_master (product_id, product_name, category_id) VALUES
('P101', 'Laptop', 'C1'),
('P102', 'Headphones', 'C1'),
('P201', 'Jacket', 'C2'),
('P202', 'Shoes', 'C2'),
('P301', 'SQL Book', 'C3');

INSERT INTO customer_master (customer_id, customer_name) VALUES
('U1', 'Aman'),
('U2', 'Priya'),
('U3', 'Rahul'),
('U4', 'Simran');

INSERT INTO order_details (order_id, customer_id, product_id, quantity, unit_price) VALUES
('O101', 'U1', 'P101', 1, 5000),
('O102', 'U2', 'P102', 2, 1000),
('O103', 'U1', 'P201', 1, 3000),
('O104', 'U3', 'P202', 2, 2000),
('O105', 'U4', 'P301', 3, 500),
('O106', 'U2', 'P101', 1, 5000),
('O107', 'U3', 'P201', 1, 3000),
('O108', 'U4', 'P102', 1, 1000);
```

Task 3:

```sql
SELECT
    c.category_id,
    c.category_name,
    SUM(o.quantity) AS total_products_sold,
    COUNT(DISTINCT o.order_id) AS total_orders,
    COUNT(DISTINCT o.customer_id) AS unique_customers,
    SUM(o.quantity * o.unit_price) AS total_revenue,
    ROUND(SUM(o.quantity * o.unit_price) / COUNT(DISTINCT o.order_id), 2) AS avg_revenue_per_order,
    CASE
        WHEN SUM(o.quantity * o.unit_price) >= 10000 THEN 'Excellent'
        WHEN SUM(o.quantity * o.unit_price) >= 6000 AND SUM(o.quantity * o.unit_price) < 10000 THEN 'Good'
        WHEN SUM(o.quantity * o.unit_price) >= 3000 AND SUM(o.quantity * o.unit_price) < 6000 THEN 'Average'
        ELSE 'Low'
    END AS category_performance
FROM category_master c
JOIN product_master p ON c.category_id = p.category_id
JOIN order_details o ON p.product_id = o.product_id
GROUP BY c.category_id, c.category_name
ORDER BY c.category_id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	category_id [PK] character varying (10)	category_name character varying (50)	total_products_sold bigint	total_orders bigint	unique_customers bigint	total_revenue numeric	avg_revenue_per_order numeric	category_performance text
1	C1	Electronics	5	4	3	13000	3250.00	Excellent
2	C2	Clothing	4	3	2	10000	3333.33	Excellent
3	C3	Books	3	1	1	1500	1500.00	Low

Task 4:

	QUERY PLAN text
1	GroupAggregate (cost=74.72..98.02 rows=440 width=276) (actual time=0.102..0.111 rows=3.00 loops=1)
2	Group Key: c.category_id
3	Buffers: shared hit=3
4	-> Sort (cost=74.72..75.84 rows=450 width=268) (actual time=0.081..0.083 rows=8.00 loops=1)
5	Sort Key: c.category_id, o.order_id
6	Sort Method: quicksort Memory: 25kB
7	Buffers: shared hit=3
8	-> Hash Join (cost=38.00..54.88 rows=450 width=268) (actual time=0.054..0.060 rows=8.00 loops=1)
9	Hash Cond: ((p.category_id)::text = (c.category_id)::text)
10	Buffers: shared hit=3
11	-> Hash Join (cost=18.10..33.79 rows=450 width=150) (actual time=0.024..0.028 rows=8.00 loops=1)
12	Hash Cond: ((o.product_id)::text = (p.product_id)::text)
13	Buffers: shared hit=2
14	-> Seq Scan on order_details o (cost=0.00..14.50 rows=450 width=150) (actual time=0.006..0.007 rows=8.00 loops=1)
15	Buffers: shared hit=1
16	-> Hash (cost=13.60..13.60 rows=360 width=76) (actual time=0.011..0.011 rows=5.00 loops=1)
17	Buckets: 1024 Batches: 1 Memory Usage: 9kB
18	Buffers: shared hit=1
19	-> Seq Scan on product_master p (cost=0.00..13.60 rows=360 width=76) (actual time=0.006..0.007 rows=5.00 loop...
20	Buffers: shared hit=1
21	-> Hash (cost=14.40..14.40 rows=440 width=156) (actual time=0.024..0.024 rows=3.00 loops=1)
22	Buckets: 1024 Batches: 1 Memory Usage: 9kB
23	Buffers: shared hit=1
24	-> Seq Scan on category_master c (cost=0.00..14.40 rows=440 width=156) (actual time=0.017..0.018 rows=3.00 loops...
25	Buffers: shared hit=1
26	Planning Time: 0.280 ms
27	Execution Time: 0.172 ms

Task 5:

```
CREATE MATERIALIZED VIEW mv_category_sales AS
SELECT
    c.category_id,
    c.category_name,
    SUM(o.quantity) AS total_products_sold,
    COUNT(DISTINCT o.order_id) AS total_orders,
    COUNT(DISTINCT o.customer_id) AS unique_customers,
    SUM(o.quantity * o.unit_price) AS total_revenue,
    ROUND(SUM(o.quantity * o.unit_price) / COUNT(DISTINCT o.order_id), 2) AS avg_revenue_per_order,
    CASE
        WHEN SUM(o.quantity * o.unit_price) >= 10000 THEN 'Excellent'
        WHEN SUM(o.quantity * o.unit_price) >= 6000 AND SUM(o.quantity * o.unit_price) < 10000 THEN 'Good'
        WHEN SUM(o.quantity * o.unit_price) >= 3000 AND SUM(o.quantity * o.unit_price) < 6000 THEN 'Average'
        ELSE 'Low'
    END AS category_performance
FROM category_master c
JOIN product_master p ON c.category_id = p.category_id
JOIN order_details o ON p.product_id = o.product_id
GROUP BY c.category_id, c.category_name;
```

Task 6:

```
107  SELECT * FROM mv_category_sales ORDER BY category_id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	category_id character varying (10)	category_name character varying (50)	total_products_sold bigint	total_orders bigint	unique_customers bigint	total_revenue numeric	avg_revenue_per_order numeric	category_performance text
1	C1	Electronics	5	4	3	13000	3250.00	Excellent
2	C2	Clothing	4	3	2	10000	3333.33	Excellent
3	C3	Books	3	1	1	1500	1500.00	Low

Task 7:

```
109  EXPLAIN ANALYZE SELECT * FROM mv_category_sales ORDER BY category_id;
```

Data Output | Messages | Notifications

Showing rows

	QUERY PLAN text
1	Sort (cost=23.03..23.68 rows=260 width=276) (actual time=0.033..0.033 rows=3.00 loops=1)
2	Sort Key: category_id
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=1
5	-> Seq Scan on mv_category_sales (cost=0.00..12.60 rows=260 width=276) (actual time=0.015..0.016 rows=3.00 loop...
6	Buffers: shared hit=1
7	Planning Time: 0.066 ms
8	Execution Time: 0.046 ms

Task 8:

Original

Planning Time: 0.218 ms

Execution Time: 0.160 ms

Materialized

Planning Time: 0.094 ms

Execution Time: 0.070 ms

Task 9:

```
INSERT INTO order_details (order_id, customer_id, product_id, quantity, unit_price)
VALUES ('O109', 'U1', 'P301', 2, 500);
```

Task 10:

```
114  SELECT * FROM mv_category_sales ORDER BY category_id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	category_id character varying (10)	category_name character varying (50)	total_products_sold bigint	total_orders bigint	unique_customers bigint	total_revenue numeric	avg_revenue_per_order numeric	category_performance text
1	C1	Electronics	5	4	3	13000	3250.00	Excellent
2	C2	Clothing	4	3	2	10000	3333.33	Excellent
3	C3	Books	3	1	1	1500	1500.00	Low

Task 11:

```
REFRESH MATERIALIZED VIEW mv_category_sales;
```

Task 12:

```
118  SELECT * FROM mv_category_sales ORDER BY category_id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	category_id character varying (10)	category_name character varying (50)	total_products_sold bigint	total_orders bigint	unique_customers bigint	total_revenue numeric	avg_revenue_per_order numeric	category_performance text
1	C1	Electronics	5	4	3	13000	3250.00	Excellent
2	C2	Clothing	4	3	2	10000	3333.33	Excellent
3	C3	Books	5	2	2	2500	1250.00	Low

LEARNING OUTCOMES:

- Mastering the integration of multiple JOINs, aggregations, and conditional CASE statements to generate comprehensive business reports.
- Creating Materialized Views in PostgreSQL to precompute and physically store the results of resource-intensive analytical queries.
- Utilizing EXPLAIN ANALYZE to benchmark execution times and validate the performance advantages of reading from pre-aggregated views.
- Managing data consistency by manually executing REFRESH MATERIALIZED VIEW to synchronize stale views with updated source tables.

CONCLUSION:

The SQL query successfully classified each node in the Tree table as Root, Inner, or Leaf
Materialized views provide a powerful optimization strategy for read-heavy database environments by caching complex query results on disk. While they drastically reduce execution times and system workloads for frequently accessed reports, they introduce a trade-off between performance and real-time accuracy, requiring deliberate database maintenance and refresh schedules to ensure reporting data remains synchronized with the underlying transactional tables.